

Computer Vision Competition Workbook

A 10-day offline field guide

Image classification · Object detection · Semantic segmentation · Instance segmentation

Built from the study notebooks and competition rules in this repository.

Purpose: ten focused study sessions without a computer or phone. Read actively, write code by hand, solve the diagnostics, and leave each day with one reusable competition decision.

Default competition stance

- Make the validation trustworthy before making the model large.
- Prefer an official architecture from an allowed library.
- Default to `weights=None`; pretrained checkpoints are optional, must be available offline, and must be legal.
- Cache validation predictions once; tune post-processing from the cache.
- Inspect per-class errors, not only one headline score.
- Treat every submission as a data-engineering artifact that must be audited.

Name: _____

Trip dates: _____

Target competition: _____

How to use this workbook

Each day is designed for roughly 60–90 minutes. No execution is required. The work alternates between recall, code tracing, metric calculation, and competition decisions.

Daily rhythm

1. **Recall — 10 minutes.** Close the workbook and write everything you remember about the topic.
2. **Study — 25 minutes.** Read the core notes and annotate them.
3. **Code by hand — 20 minutes.** Complete the skeleton without looking at the answer key.
4. **Competition drill — 20 minutes.** Solve the scenario and justify the decision.
5. **Exit ticket — 5 minutes.** Write the three things you would implement first in a real notebook.

Do not merely copy code. For every tensor, write its shape, dtype, device, and label meaning. For every metric, write what kind of failure can change it.

Ten-day route

Day	Focus	Artifact you should be able to reproduce
1	Competition framing and validation	Leakage-safe split and baseline plan
2	Image classification baseline	Dataset → model → Macro-F1 pipeline
3	Classification improvements	Imbalance, augmentation, TTA, error-analysis decision
4	Detection foundations	Box conversion, IoU, NMS, target audit
5	Detection experiment	Offline Faster R-CNN training and threshold plan
6	Semantic segmentation foundations	Mask contract, confusion matrix, per-class IoU
7	DeepLabV3 experiment	Offline training, validation, and logit ensemble plan
8	Instance-segmentation data	Instance-ID decoding and stratified folds
9	Mask R-CNN experiment	Five-loss training and prediction cache
10	OOF, ensembling, submission	End-to-end competition playbook

Repository-aligned model choices

These are strong, allowed baselines—not universal winners.

Task	Default architecture	Offline constructor principle	Primary local metric
Classification	Torchvision ResNet or compact CNN	<code>weights=None</code>	Macro-F1 + per-class recall
Detection	<code>fasterrcnn_resnet50_fpn_v2</code>	<code>weights=None, weights_backbone=None</code>	mAP / class-aware F1 at chosen IoU
Semantic segmentation	<code>deeplabv3_resnet50</code>	<code>weights=None, weights_backbone=None</code>	mean IoU + per-class IoU
Instance segmentation	<code>maskrcnn_resnet50_fpn_v2</code>	<code>weights=None, weights_backbone=None</code>	official mask AP; local mask-IoU F1

The competition loop

```
rules → data audit → split manifest → file-backed Dataset
      → smallest legal baseline → OOF prediction cache
      → metric + per-class/slice errors → one controlled change
      → fold models → test ensemble → submission audit
```

The arrow back from error analysis should change exactly one important assumption at a time. If the split, initialization, epochs, and evaluation distribution also change, the comparison no longer answers a clean question.

Day 1 — Frame the task and protect validation

Learning goal

Turn an unfamiliar computer-vision dataset into a trustworthy experiment plan before training anything.

1. Start from the score and output format

Write down these five contracts:

1. **Input unit:** one image, one crop, one video, or a group of related images?
2. **Target unit:** one class, many boxes, one class mask, or many instance masks?
3. **Metric:** what is averaged, and which errors matter equally?
4. **Submission unit:** one row per image, per class, per box, or per instance?
5. **Constraints:** allowed imports, time, memory, internet, and pretrained-weight legality.

A model can be accurate and still score zero if IDs are reordered, background labels are wrong, box coordinates use the wrong convention, or RLE flattening is reversed.

2. Identify the leakage unit

The row in a CSV is not always an independent sample. Images may share patient, product, scene, original video, camera burst, or augmented source.

Data relationship	Required split idea
Independent, single-label images	StratifiedKFold
Several images per patient/product/video	Group-aware split; stratify groups if possible
Detection or segmentation with many labels per image	Documented proxy or iterative multilabel split
Time-ordered deployment	Forward/temporal validation
Organizer gives a fixed validation set	Preserve it exactly

Non-negotiable invariants

- Every training observation appears in exactly one OOF validation fold.
- No leakage group crosses train/validation.
- Fold membership is stored by stable sample ID.
- Validation receives no oversampling and no random augmentation.
- Preprocessing learned from data is fit on training only.

3. Audit before modeling

For every task, inspect:

- missing/corrupt files;
- duplicate IDs and duplicate pixels;
- image sizes, channels, dtype, and color order;
- label mapping and background convention;
- class support by fold;
- small-object or small-region frequency;
- empty images/masks;
- train/test distribution shifts;
- annotation validity.

Task-specific annotation checks

Classification: exactly one known label per row; stable `class_name` ↔ `class_id` mapping.

Detection: $x1 < x2, y1 < y2$; boxes are inside the image; label count equals box count.

Semantic segmentation: mask size matches image size; values belong to `{0,...,C-1}` plus optional `ignore_index`.

Instance segmentation: each nonzero instance ID corresponds to exactly one class; masks are nonempty; derived boxes have positive area.

4. Baseline budget

Write the smallest experiment that can falsify your assumptions:

```
data audit:      _____ minutes
one split/fold:  _____ minutes
one train epoch: _____ minutes
full validation: _____ minutes
error report:    _____ minutes
submission smoke: _____ minutes
```

Reserve time for failures. A useful competition budget often spends more time on validation, prediction caching, and submission auditing than beginners expect.

Handwritten exercise 1A — choose the split

For each case, name the splitter and the stratification/group key.

1. 8,000 wildlife images; five frames may come from the same camera burst; single class per image.
2. 1,200 pathology tiles from 80 patients; one patient has many tiles.
3. 900 detection images; each image may contain cars, bikes, and people; many are empty.
4. Road segmentation frames recorded in chronological drives.

Your answers:

1. _____
2. _____
3. _____
4. _____

Handwritten exercise 1B — write the fold audit

Complete the invariants:

```
assert fold_table["image_id"]._____
assert set(fold_table["fold"]) == _____
assert train_ids._____(validation_ids)
assert len(train_ids | validation_ids) == _____
```

Competition drill

Your public leaderboard rises after you tune a threshold on the test predictions' class distribution. Your local OOF score is unchanged.

- What information leaked? _____
- Why is the improvement unreliable? _____
- What should select the threshold instead? _____

Exit ticket

My leakage unit is likely: _____

My primary metric and two supporting metrics are: _____

The first three audit tables I will create are: _____

Day 2 — Build the image-classification baseline

Learning goal

Reproduce the complete classification boundary: records, label mapping, transforms, model, loss, and Macro-F1.

1. Dataset contract

The file-backed Dataset should own path loading, RGB conversion, transforms, and target parsing.

```
input row: image_id, image_path, class_name, fold
output:    image float32 [C,H,W], class_id int64 scalar
range:     normally [0,1] before normalization
mapping:    class_to_id saved with the checkpoint
```

The validation transform is deterministic. A standard pair is:

```
train_tf = v2.Compose([
    v2.RandomResizedCrop((SIZE, SIZE), scale=(0.75, 1.0)),
    v2.RandomHorizontalFlip(0.5),
    v2.ColorJitter(0.15, 0.15, 0.15, 0.05),
    v2.ToImage(), v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean=MEAN, std=STD),
])

val_tf = v2.Compose([
    v2.Resize((SIZE, SIZE)),
    v2.ToImage(), v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean=MEAN, std=STD),
])
```

Use only label-preserving transforms. Horizontal flip is unsafe when left/right orientation changes the class.

2. Model boundary

For an offline competition, start with an official architecture and random weights:

```
model = resnet18(weights=None)
model.fc = nn.Linear(model.fc.in_features, num_classes)
```

If a legal checkpoint is attached, use it explicitly and record its provenance. Do not let **DEFAULT** trigger an unexpected network download.

For small data, compare:

- frozen backbone + new head;
- later unfreeze the final block with a lower learning rate;
- full random-initialization training if no checkpoint exists.

3. Train/evaluate semantics

```
model.train()
for images, labels in train_loader:
    logits = model(images.to(device)) # [B,C]
    loss = criterion(logits, labels.to(device))
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

model.eval()
with torch.inference_mode():
    logits = model(images.to(device))
    probabilities = logits.softmax(dim=1)
```

CrossEntropyLoss consumes raw logits and **int64 [B]** class indices. Do not apply softmax before the loss.

4. Metric hierarchy

For class **c**:

```
precision_c = TP_c / (TP_c + FP_c)
recall_c    = TP_c / (TP_c + FN_c)
F1_c        = 2 * precision_c * recall_c / (precision_c + recall_c)
Macro-F1     = mean(F1_c over classes)
```

Accuracy weights common classes more heavily. Macro-F1 gives each class one equal vote, even when support differs.

Always print:

- validation support per class;
- accuracy and Macro-F1;
- per-class precision/recall/F1;
- confusion matrix;
- OOF row count and unique ID count.

Handwritten exercise 2A — finish the Dataset

```
class ImageDataset(Dataset):
    def __getitem__(self, index):
        row = self.rows.iloc[index]
        image = Image.open(row["_____"]).convert("____")
        image = self.transform(image)
        label = torch.tensor(
            self.class_to_id[row["_____"]], dtype=torch._____
        )
        return _____, _____
```

Write the output shape/dtype:

Image: ____ **Label:** ____

Handwritten exercise 2B — confusion matrix

Ground truth: [0, 0, 1, 1, 1, 2]

Prediction: [0, 1, 1, 1, 2, 2]

Using rows=true and columns=predicted, fill:

True \ Pred	0	1	2
0			
1			
2			

Which class has the weakest recall? ____

Competition drill

Training loss falls, validation accuracy rises, but Macro-F1 falls. List two plausible causes and one diagnostic for each.

1. Cause: ____ **Diagnostic:** ____
2. Cause: ____ **Diagnostic:** ____

Exit ticket

Write from memory the six objects needed for a baseline:

records → ____ → ____ → ____ → ____ → metrics

Day 3 — Improve classification without fooling yourself

Learning goal

Choose among weighted loss, weighted sampling, augmentation, TTA, and error-driven changes using controlled evidence.

1. Imbalance strategies

Stratification is not balancing. It preserves class proportions in each fold. Balancing changes training exposure or loss contribution.

Weighted loss

Compute counts from the training fold only. A common balanced weight is:

```
w_c = N / (C * count_c)
```

```
weights = compute_class_weight("balanced", classes=class_ids, y=train_labels)
criterion = nn.CrossEntropyLoss(
    weight=torch.tensor(weights, dtype=torch.float32, device=device)
)
```

Risks: noisy rare labels gain influence; huge weights can destabilize training.

Weighted sampling

Convert class weights to one weight per training observation:

```
sample_weights = class_weights[train_labels]
sampler = WeightedRandomSampler(
    sample_weights, num_samples=len(train_labels), replacement=True
)
```

Use `sampler=...`, not `shuffle=True`. Keep observations per epoch equal when comparing with the baseline.

Risks: repeated minority examples overfit; the observed training distribution no longer matches validation.

Decision rule

- Start with plain CE.
- Try weighted CE when rare-class misses dominate.
- Try sampling when minority examples rarely reach batches.
- Do not combine both blindly; the correction may be doubled.
- Never rebalance validation.

2. Controlled experiment table

Fill one row per run:

Run	Split ID	Init seed	Epochs	Images/epoch	Main change	Macro-F1	Rare recall
Baseline					plain CE		
Weighted loss					loss only		
Sampler					loader only		

If more than the “main change” differs, the comparison is observational, not controlled.

3. Error analysis loop

Create one OOF record per image:

```
image_id, fold, true_id, pred_id, confidence,
probabilities, loss, metadata slices
```

Then inspect:

1. highest-confidence wrong predictions;
2. top confusion pairs;
3. low-confidence correct predictions;

4. performance by blur, brightness, size, source, or camera;
5. suspected label errors and duplicates.

Map errors to changes:

Error pattern	Candidate change
Orientation sensitivity	Safe flip/rotation augmentation
Tiny discriminative region	Larger input or crop policy
Rare class ignored	Weighted loss or sampler
Similar classes confused	More resolution, targeted examples, label audit
Confidence too high	Calibration, label smoothing, stronger validation
Train good, validation poor	regularization, more data, simpler model, leakage audit

4. Test-time augmentation

TTA is an inference ensemble. Apply label-preserving transforms, reverse spatial transforms if needed, and average comparable outputs.

For classification:

```
p = 0.5 * softmax(model(x)) + 0.5 * softmax(model(horizontal_flip(x)))
```

Measure both score and latency. Never assume TTA helps.

Handwritten exercise 3A — calculate weights

Training counts are [80, 16, 4], so $N=100$, $C=3$.

$w_0 = _$ $w_1 = _$ $w_2 = _$

Which class contributes most per example? $_$

Handwritten exercise 3B — choose a strategy

Minority recall is 0.10. Weighted loss raises it to 0.48 but lowers majority recall from 0.92 to 0.55. Sampler gives minority recall 0.39 and majority recall 0.86. Macro-F1 values are 0.44, 0.56, and 0.63 respectively.

Which run is the current winner? $_$ **Why?** $_$

What evidence would make you reject it? $_$

Competition drill

Your TTA score is +0.002 Macro-F1 but inference is $2.1\times$ slower and the time limit is tight. Write the decision:

Exit ticket

My default imbalance sequence is: $_$

My top three error-analysis slices are: $_$

Day 4 — Object-detection foundations

Learning goal

Audit detection targets and reason correctly about box formats, IoU, NMS, and class-aware matching.

1. Box conventions

XYXY: (x1, y1, x2, y2) corners in pixels.

XYWH: (x, y, width, height); confirm whether (x,y) is top-left or center.

YOLO normalized: (class, cx/W, cy/H, width/W, height/H).

YOLO normalized to pixel XYXY:

```
cx = cx_n·W      cy = cy_n·H
bw = w_n·W      bh = h_n·H
x1 = cx - bw/2   y1 = cy - bh/2
x2 = cx + bw/2   y2 = cy + bh/2
```

Clip to the image only after understanding why coordinates are outside. Silent clipping can hide annotation bugs.

2. IoU

```
intersection width = max(0, min(x2A,x2B) - max(x1A,x1B))
intersection height = max(0, min(y2A,y2B) - max(y1A,y1B))
IoU = intersection / (areaA + areaB - intersection)
```

IoU is 0 for no overlap and 1 for identical boxes. A higher evaluation threshold requires tighter localization.

3. Non-maximum suppression

NMS removes duplicate predictions, not ground truth.

1. Filter low scores.
2. Process each class separately.
3. Keep the highest score.
4. Suppress remaining same-class boxes with IoU above the NMS threshold.
5. Repeat.

Low NMS threshold suppresses aggressively and may delete nearby objects. High threshold preserves objects but allows duplicates.

4. Torchvision target contract

For one image:

```
target = {
  "boxes": float32[N,4], # XYXY
  "labels": int64[N], # foreground starts at 1
  "image_id": int64[1],
  "area": float32[N],
  "iscrowd": int64[N],
}
```

Images are a list of `float32 [C,H,W]` tensors in `[0,1]`. A custom collator returns a tuple/list of images and targets because `N` varies.

Use `tv_tensors.BoundingBoxes` and Torchvision `v2` transforms so geometric augmentation updates boxes with the image.

5. Detection validation proxy

Single-label stratification does not directly fit multi-object images. Possible documented proxies:

- rarest class present;
- sorted class-presence signature;
- dominant class + object-count bin;
- empty/nonempty + object-count bin.

If images share scenes/videos/patients, group separation outranks perfect proxy balance.

Handwritten exercise 4A — convert a box

Image is $W=200, H=100$. YOLO row is $class=2, cx_n=.50, cy_n=.40, w_n=.20, h_n=.30$.

Pixel center: (____, ____)

Pixel size: (____, ____)

XYXY: (____, ____, ____, ____)

Handwritten exercise 4B — IoU

Box A $(0,0,10,10)$, Box B $(5,0,15,10)$.

Intersection area: __ **Union area:** _ **IoU:** __

Handwritten exercise 4C — NMS

Same class predictions:

- A score .90, overlaps B at .70
- B score .80, overlaps C at .10
- C score .60, overlaps A at .05

At NMS IoU $.50$, kept boxes are: _____

What changes if B belongs to a different class? _____

Competition drill

Boxes look correct on 10 images, but mAP is near zero. List four contract errors to test before changing the model.

1. ____ **2.** ____

2. ____ **4.** ____

Exit ticket

Write the detection target keys from memory: _____

Day 5 — Run a real detection experiment

Learning goal

Design an offline Faster R-CNN experiment whose losses, prediction cache, and threshold choice answer useful questions.

1. Why Faster R-CNN ResNet50-FPN V2

The repository uses Torchvision's official V2 builder as an accuracy-first general detector. It has:

- ResNet50 feature extractor;
- FPN multi-scale feature maps;
- RPN object proposals;
- ROI classification and box regression heads.

For restricted/offline use:

```
model = fasterrcnn_resnet50_fpn_v2(  
    weights=None,  
    weights_backbone=None,  
    num_classes=NUM_CLASSES,      # includes background  
    min_size=IMAGE_SIZE,  
    max_size=IMAGE_SIZE,  
)
```

Random initialization is slower to learn, but the API and target boundary stay competition-relevant.

2. Training-mode contract

```
model.train()  
losses = model(images, targets)
```

Expected losses:

- `loss_classifier` — ROI class prediction;
- `loss_box_reg` — ROI localization;
- `loss_objectness` — RPN foreground/background;
- `loss_rpn_box_reg` — RPN proposal localization.

Sum them, reject non-finite values, backpropagate, and record each component separately.

3. Evaluation-mode contract

```
model.eval()  
with torch.inference_mode():  
    outputs = model(images)
```

Each output contains `boxes [P,4]`, `labels [P]`, and `scores [P]` on the model device. Cache them on CPU with the stable image ID and ground truth.

4. Small-object tuning surface

FPN defaults can be poor for tiny custom objects. Inspect:

- input resolution;
- anchor sizes and aspect ratios;
- RPN proposals before/after NMS;
- RPN NMS threshold;
- detection score floor;
- ROI NMS threshold;
- detections per image.

Change one coherent hypothesis at a time. Example: "objects are smaller than the smallest useful default anchor" justifies smaller anchors and perhaps more retained proposals.

5. Evaluation and threshold sweep

For each class:

1. sort predictions globally or per image according to the metric;

2. match only same-class boxes;
3. match each ground truth at most once;
4. unmatched prediction = FP;
5. unmatched ground truth = FN.

Cache predictions at a low model score floor. Sweep an operating threshold from the cache. Report precision, recall, F1, mAP50, and mAP75 where applicable. Use the official competition scorer for final selection.

Handwritten exercise 5A — fill the update

```
model._____(  
loss_dict = model(device_images, device_targets)  
loss = _____  
optimizer._____(  
loss._____(  
optimizer._____(
```

Why must targets move to the same device? _____

Handwritten exercise 5B — diagnose losses

After five epochs:

- classifier loss falls quickly;
- objectness falls;
- box losses remain flat;
- qualitative boxes are badly shifted.

Write three checks before increasing epochs:

1. _____
2. _____
3. _____

Handwritten exercise 5C — threshold tradeoff

Score threshold	Precision	Recall	F1
.10	.42	.91	
.30	.66	.73	
.60	.88	.40	

Calculate the three F1 values. Which threshold would you choose if the metric is F1? _____

Competition drill

Validation has only two examples of the rare class. mAP varies wildly by fold. Write a better data/validation response—not a model response.

Exit ticket

The four detection losses are: _____

The three small-object controls I will inspect first are: _____

Day 6 — Semantic-segmentation foundations

Learning goal

Reason about dense targets, DeepLab outputs, ignore regions, and IoU from first principles.

1. Semantic target contract

Semantic segmentation assigns one class to each pixel, but does not separate two objects of the same class.

```
image batch: float32 [N,3,H,W]
logits:      float32 [N,C,H,W]
target mask: int64   [N,H,W]
prediction:  argmax(logits, dim=1) → int64 [N,H,W]
```

`CrossEntropyLoss` consumes raw logits and class-index targets. Do not one-hot encode the target for ordinary multiclass segmentation.

An optional `ignore_index` excludes void/unlabeled pixels from loss and metrics. Background is usually a real class, not automatically ignored.

2. DeepLabV3 mental model

DeepLabV3 combines a backbone with atrous/dilated convolution and an ASPP classifier to collect context at several receptive fields.

Torchvision returns a dictionary:

```
outputs = model(images)
main_logits = outputs["out"]
aux_logits = outputs.get("aux")
```

The auxiliary head helps training; use the main output for final inference.

Offline builder:

```
model = deeplabv3_resnet50(
    weights=None,
    weights_backbone=None,
    num_classes=NUM_CLASSES,
    aux_loss=True,
)
```

If adapting a pretrained model, replace both main and auxiliary final classifiers.

3. Pixel confusion and IoU

For class `c`:

```
intersection_c = pixels where prediction=c AND target=c
union_c        = pixels where prediction=c OR target=c
IoU_c          = intersection_c / union_c
mean IoU       = mean over documented classes
```

Pixel accuracy may look excellent when background dominates. Always report foreground IoUs and class support.

If a class is absent from both prediction and target, define the competition's convention: ignore it, treat as perfect, or treat as undefined. Do not hide this choice.

4. Geometric transforms

Image and mask must receive identical spatial transforms. Image interpolation may be bilinear; class masks require nearest-neighbor interpolation to preserve integer IDs.

Photometric transforms apply only to the image. Validation remains deterministic.

Handwritten exercise 6A — shapes

For batch size 4, five classes, and 256×320 images:

- logits shape: _____
- target shape/dtype: _____
- prediction shape/dtype: _____

Which dimension does `argmax` reduce? _____

Handwritten exercise 6B — IoU

For class 1: TP pixels = 30, FP pixels = 10, FN pixels = 20.

Intersection = __ **Union** = _ **IoU** = __

Pixel accuracy is .94 but class-1 IoU is your answer above. What does this suggest?

Handwritten exercise 6C — find the bug

```
mask = resize(mask, interpolation=BILINEAR)
target = torch.tensor(mask, dtype=torch.float32)
loss = cross_entropy(logits.softmax(1), target)
```

Circle three errors and rewrite the correct intent:

Competition drill

A class occupies less than 0.1% of pixels. Name one validation change, one loss/data change, and one modeling/resolution change worth testing.

Validation: _____

Loss/data: _____

Model/resolution: _____

Exit ticket

Write the semantic tensor contract from memory: _____

Day 7 — Train and compare DeepLabV3 models

Learning goal

Design a fair semantic-segmentation experiment with offline initialization, parameter groups, per-class IoU, and aligned logit ensembling.

1. Head training and fine-tuning

A staged plan:

1. Build the official architecture with no checkpoint download.
2. If using a legal checkpoint, replace main and auxiliary classifiers.
3. Freeze the backbone for head-only training.
4. Unfreeze the final backbone stage with a lower learning rate.
5. Compare on the same folds, transforms, epochs, and input resolution.

Parameter groups express intent:

```
optimizer = torch.optim.SGD([
    {"params": model.classifier.parameters(), "lr": 1e-2},
    {"params": model.aux_classifier.parameters(), "lr": 5e-3},
], momentum=0.9)
```

When the backbone is frozen, keep its BatchNorm behavior controlled; `model.train()` can otherwise update running statistics even if gradients are disabled.

2. Loss

```
outputs = model(images)
main_loss = F.cross_entropy(outputs["out"], masks, ignore_index=IGNORE)
aux_loss = F.cross_entropy(outputs["aux"], masks, ignore_index=IGNORE)
loss = main_loss + 0.4 * aux_loss
```

Class-weighted loss or a region-aware sampling strategy can help rare classes, but validate on the untouched distribution and inspect false-positive expansion.

3. ResNet50 versus ResNet101

Compare with identical preprocessing and batch:

- parameter count;
- latency per image;
- peak memory;
- mean and per-class IoU;
- fold stability.

The larger backbone is justified only when the metric gain fits the runtime/memory budget.

4. Segmentation ensemble

For aligned models and classes:

```
ensemble_logits = torch.stack([logits_fold0, logits_fold1, logits_fold2]).mean(0)
prediction = ensemble_logits.argmax(1)
```

Average logits or probabilities consistently before `argmax`. If spatial sizes differ, resize logits to the original evaluation size first. Never average hard class-ID masks.

5. Error slices

Inspect IoU by:

- class;
- object/region size;
- image brightness or weather;
- boundary distance;
- empty versus nonempty mask;
- source group.

Boundary failures may need more resolution or boundary-aware post-processing; global class confusion may need label or feature changes.

Handwritten exercise 7A — replace both heads

```
model.classifier[-1] = nn.Conv2d(256, _____, 1)
model.aux_classifier[-1] = nn.Conv2d(256, _____, 1)
```

Does `num_classes` include background? __ **Why?** _____

Handwritten exercise 7B — fair comparison

Circle every unfair difference:

- ResNet50 uses 256×256, 20 epochs, fold 0.
- ResNet101 uses 512×512, 35 epochs, fold 1.
- Both use the same metric function.
- ResNet101 uses TTA and ResNet50 does not.

Rewrite a fair first comparison:

Handwritten exercise 7C — model choice

Model	mIoU	Rare IoU	ms/image	Memory
R50	.612	.301	38	2.7 GB
R101	.621	.319	71	4.5 GB

If the time limit allows at most 45 ms/image, choose: ____

If offline inference has no time penalty, what else must you verify before choosing R101?

Competition drill

An ensemble improves mIoU but destroys a thin rare class. Write two diagnostics and one class-aware remedy.

Exit ticket

Main output key: __ **Auxiliary output key:** _ *Target dtype:* ____

Day 8 — Build instance-segmentation data and folds

Learning goal

Convert integer instance-ID masks into the full Mask R-CNN target schema and persist appropriate folds.

1. Instance versus semantic masks

Suppose two touching cars both have class “car.”

- Semantic mask: both regions contain the same class ID.
- Instance mask: the first region has instance ID 1, the second instance ID 2; a separate mapping gives both class “car.”

A common file pair is:

image.png	RGB image
instance.png	0=background, 1..N=instance IDs
labels.json	labels ordered by instance ID

Do not assume the instance ID is the class ID.

2. Decode target

```
instance_ids = [v for v in np.unique(instance_map) if v != 0]
masks = torch.stack([
    torch.from_numpy((instance_map == instance_id).astype(np.uint8))
    for instance_id in instance_ids
])
labels = torch.tensor(instance_labels, dtype=torch.int64)
boxes = masks_to_boxes(masks)
```

Complete target:

```
target = {
    "boxes": float32[N,4],
    "labels": int64[N],
    "masks": uint8[N,H,W],
    "image_id": int64[1],
    "area": float32[N],
    "iscrowd": int64[N],
}
```

Check `len(boxes) == len(labels) == len(masks)`.

3. Synchronized transforms

Wrap structures as `tv_tensors.Image`, `tv_tensors.Mask`, and `tv_tensors.BoundingBoxes`. Apply geometric transforms jointly. Convert to pure tensors at the model boundary.

An image flip must update both masks and boxes. Color jitter affects only the image.

4. Stratification proxy

Instance-segmentation images are usually multilabel and have varying object counts. A compact proxy used in the repository is:

```
class composition + instance-count bin
examples: circle_1, square_1, mixed_2
```

Real alternatives include rarest class present, class-presence signature, and instance-count/area bins. If samples share scenes or patients, use group-aware splitting even when class balance becomes imperfect.

Persist the table:

```
image_id, image_path, mask_path, split_key, group_id, fold
```

Handwritten exercise 8A — decode a map

Instance map values are `{0, 3, 7}`. Label mapping is `{3: 2, 7: 2}`.

- Number of instances: _____
- `masks` shape for 128×128 image: _____
- `labels`: _____

- Are the two objects separate at inference? _____

Handwritten exercise 8B — target audit

Fill the dtype/shape:

Key	Shape	Dtype
boxes		
labels		
masks		
image_id		
area		
iscrowd		

Handwritten exercise 8C — fold design

You have 300 images from 30 videos. Each image may contain three classes. Write the priority order for a valid split.

1. _____
2. _____
3. _____

Competition drill

A horizontal flip changes image and boxes, but not masks. Training still runs. Predict two symptoms.

1. _____
2. _____

Exit ticket

Write the six instance-target keys from memory: _____

Day 9 — Train Mask R-CNN from scratch

Learning goal

Understand the official model boundary, five losses, offline initialization, and validation-cache workflow.

1. Why Mask R-CNN ResNet50-FPN V2

Mask R-CNN extends Faster R-CNN with a per-instance mask head. The repository selects Torchvision's V2 builder as the accuracy-first allowlisted baseline. Official metadata reports stronger COCO mask accuracy than the older builder, but V2 is compute-heavy.

Offline default:

```
model = maskrcnn_resnet50_fpn_v2(  
    weights=None,  
    weights_backbone=None,  
    num_classes=NUM_CLASSES, # includes background  
    min_size=IMAGE_SIZE,  
    max_size=IMAGE_SIZE,  
)
```

If legal pretrained weights exist, load explicitly and replace **both** box and mask predictors for the target classes.

2. Training-mode output

With images and targets, the model returns:

- `loss_classifier`;
- `loss_box_reg`;
- `loss_mask`;
- `loss_objectness`;
- `loss_rpn_box_reg`.

```
loss_dict = model(device_images, device_targets)  
total_loss = sum(loss_dict.values())
```

Record every component, images seen, batches, and runtime. Reject non-finite totals.

3. Evaluation-mode output

Without targets, one result per image contains:

```
boxes    float32[P,4]  
labels   int64[P]  
scores   float32[P]  
masks    float32[P,1,H,W] # soft probabilities
```

Do not threshold masks inside the cache. Store raw scores and mask probabilities on CPU so threshold sweeps are cheap.

4. Fold training discipline

For each fold:

1. reset to the same documented initialization policy;
2. train on all rows where `fold != k`;
3. validate only rows where `fold == k`;
4. save `model_fold_k.pt`, config, label mapping, and fold table hash;
5. write OOF predictions only for fold `k`;
6. verify every training image appears once across OOF caches.

Test inference runs every fold model, then associates/ensembles predictions by image and class.

5. Failure diagnosis

Symptom	First checks
<code>loss_mask</code> flat	masks binary/nonempty; labels aligned; transforms synchronized
box loss high	coordinate format; derived boxes; image resizing
no predictions	score floor; undertraining; RPN proposals; label/background mapping
many duplicates	score threshold; NMS; proposal counts
merged touching objects	mask resolution, proposal association, instance labels
CPU too slow	smaller image, fewer proposals, bounded smoke run, simpler baseline

Handwritten exercise 9A — complete the builder policy

```
model = maskrcnn_resnet50_fpn_v2(  
    weights=_____,  
    weights_backbone=_____,  
    num_classes=_____,  
)
```

Why are both weight arguments explicit? _____

Handwritten exercise 9B — model modes

Fill the calls:

Training: `model._____()`; `result = model(_____, _____)`

Inference: `model._____()`; `result = model(_____)` inside _____

Training result type: __ *Inference result type*: _____

Handwritten exercise 9C — loss reasoning

Epoch 1 → 5: `loss_mask` drops from 1.2 to .3, but `loss_classifier` remains 1.1 and predicted labels are random.

What improved? _____

What remains broken? _____

Name two checks: _____

Competition drill

A legal pretrained checkpoint exists but is not cached, and competition internet is disabled. What is the correct plan before and during the event?

Exit ticket

Write all five losses: _____

Day 10 — OOF evaluation, ensembling, and submission

Learning goal

Finish the competition loop: cache predictions, tune without leakage, ensemble by task, and audit submission artifacts.

1. One-to-one instance matching

For local instance F1 at a fixed mask-IOU threshold:

1. choose a class;
2. filter predictions by score;
3. threshold soft masks;
4. sort predictions by score;
5. compute pairwise IOU against same-class targets;
6. greedily match the best unmatched target above the IOU threshold;
7. unmatched predictions are FP; unmatched targets are FN.

This is useful for threshold decisions but may not reproduce COCO AP. Final model selection must use the official competition evaluator.

2. Threshold sweeps

Tune from complete OOF caches:

- classification: decision/calibration threshold if multilabel or binary;
- detection: confidence and NMS thresholds;
- semantic segmentation: usually argmax; perhaps class-specific probability/post-processing thresholds;
- instance segmentation: confidence, mask binarization, and mask-NMS/merge thresholds.

Record the number of predicted instances/boxes as well as the score. A threshold can improve precision by deleting almost everything.

3. Task-appropriate fold ensembling

Task	Ensemble rule
Classification	Average aligned logits or probabilities
Detection	Class-aware NMS or box fusion across fold outputs
Semantic segmentation	Resize/aligned logits, then average before argmax
Instance segmentation	Associate same-class overlapping instances, then score-weight mask probabilities

For instance masks, never average all pixels from unrelated objects. Cluster candidates only when class and overlap support the same-instance hypothesis.

4. RLE reminder

A common mask submission uses one-indexed `start` `length` runs after column-major flattening:

```
pixels = mask.T.reshape(-1)
```

But competitions differ. Confirm:

- row-major versus column-major;
- zero- versus one-indexed starts;
- binary threshold;
- one row per image or instance;
- representation of empty masks;
- class and score columns;
- required sorting.

5. Final submission audit

- ☐ exact required columns and order
- ☐ expected row count or valid variable-row rule
- ☐ every test ID present; no unknown ID
- ☐ no duplicate rows unless instances require them
- ☐ no NaN/Inf
- ☐ labels inverse-mapped correctly
- ☐ boxes clipped and ordered correctly
- ☐ masks/RLE decode to intended shape
- ☐ probabilities/scores in valid range
- ☐ test order preserved
- ☐ smoke-submit file written and reopened

Handwritten exercise 10A — choose the ensemble

Write the rule for each output:

Classification probabilities: _____

Semantic logits: _____

Detection boxes: _____

Instance masks: _____

Handwritten exercise 10B — leakage audit

Mark valid (V) or invalid (I):

1. Select score threshold on complete OOF predictions. ____
2. Select mask threshold on public leaderboard response. ____
3. Fit normalization statistics on train plus test. ____
4. Average three fold models on test. ____
5. Use fold 0 model to predict fold 0 validation after training on all folds. ____

Handwritten exercise 10C — mock final hour

You have 60 minutes. Assign minutes:

rerun best folds/checkpoints:	_____
test inference:	_____
ensemble/post-process:	_____
submission audit:	_____
buffer/recovery:	_____
TOTAL:	60

What action will you refuse to start in the final 15 minutes? _____

Final competition drill

Write your end-to-end plan in no more than twelve lines:

1. _____
2. _____
3. _____
4. _____
5. _____
6. _____
7. _____
8. _____
9. _____
10. _____

11. _____

12. _____

Cross-task reference sheets

Tensor and target contracts

Task	Model input	Training target	Model output
Classification	[B,3,H,W] float32	[B] int64	[B,C] logits
Detection	list of [3,H,W]	list of dicts with boxes/labels	list of boxes/labels/scores
Semantic segmentation	[B,3,H,W] float32	[B,H,W] int64	dict with [B,C,H,W] logits
Instance segmentation	list of [3,H,W]	boxes/labels/masks dict	boxes/labels/scores/soft masks

Background conventions

- Classification: usually no implicit background class.
- Torchvision detection/instance segmentation: num_classes includes background at index 0; foreground labels start at 1.
- Semantic segmentation: background is normally class 0 and is included in C, unless the task specifies otherwise.

Mode and gradient checklist

```
training:  model.train() → forward with targets if detection → loss → backward → step
validation: model.eval() → torch.inference_mode() → cache raw outputs on CPU
```

BatchNorm and dropout change behavior between modes. Freezing parameters does not automatically freeze BatchNorm running statistics.

Split decision tree

```
Do observations share a patient/video/product/scene?
├ yes → group-aware split first
└ no
  └ one class per sample → StratifiedKFold
    └ multiple labels/objects → documented multilabel/task proxy

Is deployment temporal?
└ yes → preserve time direction, even if class balance is imperfect
```

Metric decision table

Symptom hidden by headline metric	Add this report
Majority class dominates accuracy	Macro-F1, per-class recall
Detection localization is loose	mAP75 or higher-loU metrics
Background dominates pixels	per-class IoU, foreground mIoU
Duplicated instances	precision, count error, mask NMS audit
Fold instability	fold mean, standard deviation, worst fold
Threshold deletes outputs	predicted count and recall

Reproducibility record

Write this beside every serious run:

```
run_id:  
git/data version:  
fold table/hash:  
seed:  
model + weight provenance:  
image size:  
train/validation transforms:  
optimizer, LR, weight decay, scheduler:  
epochs and observations/epoch:  
main metric + per-class metrics:  
inference thresholds/TTA:  
runtime and memory:  
checkpoint filename:
```

Code-from-memory sheets

Classification skeleton

```
import os
import pandas as pd
import torch
from PIL import Image
from torch import nn
from torch.utils.data import DataLoader, Dataset
from torchvision.models import resnet18
from torchvision.transforms import v2

class ImageDataset(Dataset):
    def __init__(self, frame, transform):
        self.frame = frame.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.frame)

    def __getitem__(self, index):
        row = self.frame.iloc[index]
        image = Image.open(row.image_path).convert("RGB")
        return self.transform(image), torch.tensor(row.class_id, dtype=torch.int64)

model = resnet18(weights=None)
model.fc = nn.Linear(model.fc.in_features, NUM_CLASSES)
criterion = nn.CrossEntropyLoss()
```

Write the missing training/evaluation loop on this page:

Detection skeleton

```
def collate_detection(batch):
    images, targets = zip(*batch)
    return tuple(images), tuple(targets)

model = fasterrcnn_resnet50_fpn_v2(
    weights=None,
    weights_backbone=None,
    num_classes=NUM_CLASSES,
)

model.train()
for images, targets in loader:
    images = [image.to(device) for image in images]
    targets = [{k: v.to(device) for k, v in target.items()} for target in targets]
    losses = model(images, targets)
    loss = sum(losses.values())
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Write the evaluation/cache loop:

Semantic-segmentation skeleton

```
model = deeplabv3_resnet50(
    weights=None,
    weights_backbone=None,
    num_classes=NUM_CLASSES,
    aux_loss=True,
)

outputs = model(images)
main_loss = nn.functional.cross_entropy(outputs["out"], masks)
aux_loss = nn.functional.cross_entropy(outputs["aux"], masks)
loss = main_loss + 0.4 * aux_loss
prediction = outputs["out"].argmax(dim=1)
```

Write the per-class IoU loop:

Instance-segmentation skeleton

```
instance_ids = [v for v in np.unique(instance_map) if v != 0]
masks = torch.stack([
    torch.from_numpy((instance_map == v).astype(np.uint8))
    for v in instance_ids
])
boxes = masks_to_boxes(masks)

target = {
    "boxes": boxes.to(torch.float32),
    "labels": labels.to(torch.int64),
    "masks": masks.to(torch.uint8),
    "image_id": image_id.to(torch.int64),
    "area": masks.flatten(1).sum(1).to(torch.float32),
    "iscrowd": torch.zeros(len(masks), dtype=torch.int64),
}
```

Write the offline Mask R-CNN builder and five-loss update:

Emergency debugging guide

If training crashes immediately

1. Print one sample and one batch.
2. Check every tensor's shape, dtype, min/max, device.
3. Check label range against output channels.
4. Run one forward pass without optimizer logic.
5. Check loss is finite.
6. Overfit one or two samples only as a structural diagnostic.

If score is suspiciously high

- duplicated train/validation image;
- shared patient/video/product across folds;
- target leaked into filename or pixels;
- validation transform uses train-time artifact;
- OOF row predicted by a model that trained on it;
- public leaderboard overfitting.

If score is suspiciously low

- wrong RGB/BGR or normalization;
- wrong label inverse mapping;
- background/index offset;
- wrong box format or image scale;
- bilinear interpolation on class masks;
- wrong mask threshold/RLE order;
- model left in training mode;
- checkpoint and architecture mismatch;
- test IDs reordered.

If loss decreases but metric does not

- optimized loss poorly aligned with metric;
- threshold/post-processing wrong;
- majority/background dominates loss;
- localization/mask quality below evaluation IoU;
- label noise;
- validation support too small;
- underfit rare classes or overfit common classes.

The five-print rule

Before long training, print:

1. one raw record;
2. one transformed image/target;
3. one batch contract;
4. one model output/loss dictionary;
5. one decoded prediction beside ground truth.

Answer key and discussion notes

Day 1

1A. (1) Group-aware by camera burst, with class balancing at group level if possible. (2) Group by patient; never split tiles from one patient across folds. (3) A documented multi-object proxy such as class-presence signature + empty/object-count bin; group-aware if a scene relationship exists. (4) Temporal/drive-level validation preserving future direction.

1B. `is_unique`; `{0,1,...,K-1}`; `isdisjoint`; total unique observations.

Drill. Test-distribution information influenced a validation decision. Public-leaderboard feedback is noisy and encourages overfitting. Select the threshold from complete OOF predictions using the official/local metric.

Day 2

2A. `image_path`, `RGB`, `class_name`, `int64`, `image`, `label`. Image is `float32 [3,H,W]` after the shown transform; label is scalar `int64`.

2B. Matrix rows: `[1,1,0]`, `[0,2,1]`, `[0,0,1]`. Class 0 recall is $1/2$; class 1 recall is $2/3$; class 2 recall is 1 . Class 0 is weakest.

Drill. Plausible causes include majority-class improvement hiding rare-class collapse, or threshold/calibration changes. Inspect per-class recall/F1, confusion pairs, support, and confidence distributions.

Day 3

3A. $w_0=100/(3 \cdot 80)=0.4167$; $w_1=2.0833$; $w_2=8.3333$. Each class receives similar aggregate weight if all examples contribute, while each class-2 example has the greatest individual influence.

3B. The sampler run has the largest stated Macro-F1 (0.63) and preserves majority recall better. Reject it if fold stability is poor, minority examples are memorized, inference slices fail, or the competition metric/cost differs.

TTA drill. Skip TTA if it risks timeout for a negligible, unstable gain; or use it only for a subset/model if a measured runtime budget remains.

Day 4

4A. Center `(100,40)`, size `(40,30)`, `XYXY (80,25,120,55)`.

4B. Intersection 50 , union 150 , IoU $1/3 \approx .333$.

4C. Keep A, suppress B, keep C. If B is a different class, class-aware NMS keeps A, B, and C.

Drill. Check label offset/background, `XYXY` versus `XYWH`, normalized versus pixel scale, image width/height order, prediction/ground-truth ID alignment, and evaluator IoU/class matching.

Day 5

5A. `train`; `sum(loss_dict.values())`; `zero_grad`; `backward`; `step`. Images, targets, and parameters must share a device for tensor operations.

5B. Audit annotation/coordinate conversion, transform synchronization, box clipping and positive area, image resizing scale, label mapping, and proposal/anchor fit before merely training longer.

5C. F1 values are approximately $.575$, $.694$, $.550$; choose $.30$ for this table if F1 is the target.

Drill. Improve validation support: more folds only if each has support, merge rare strata, use repeated/group-aware folds, collect more examples, and report uncertainty. Do not claim stable rare-class quality from two observations.

Day 6

6A. Logits `[4,5,256,320]`; target `int64 [4,256,320]`; prediction `int64 [4,256,320]`; reduce class dimension 1 .

6B. Intersection 30 , union $30+10+20=60$, IoU $.50$. High pixel accuracy likely reflects background dominance.

6C. Use nearest-neighbor mask resize, target `int64`, and raw logits in cross entropy: `loss = cross_entropy(logits, target)`.

Day 7

7A. Both blanks are `num_classes`. Semantic segmentation includes background if it is a documented class ID.

7B. Image size, epochs, fold, and TTA all differ. First compare on the same fold(s), resolution, training budget, transforms, seed policy, and inference procedure.

7C. Choose R50 under the 45 ms limit. Without a time penalty, verify memory, fold stability, class-level gains, checkpoint legality/availability, and whether the extra compute fits training constraints.

Day 8

8A. Two instances; masks `[2,128,128]`; labels `[2,2]`; yes, the two instance channels remain separate despite sharing a class.

8B. Boxes `[N,4] float32`; labels `[N] int64`; masks `[N,H,W] uint8`; image ID `[1] int64`; area `[N] float32`; iscrowd `[N] int64`.

8C. First prevent video leakage by grouping. Then balance class presence/rarest class across video groups. Then balance instance-count/area bins where feasible. Audit the resulting support rather than assuming success.

Drill. Masks disagree spatially with boxes/images, so mask loss may stay high or learn contradictory targets; predicted masks may be mirrored/shifted even when boxes look plausible.

Day 9

9A. `None`, `None`, target `num_classes`. Both are explicit because otherwise a builder/version may initialize the backbone from a remote checkpoint.

9B. Training: `train`, images, targets. Inference: `eval`, images, `torch.inference_mode()`. Training returns a loss dictionary; inference returns a list of prediction dictionaries.

9C. Mask-shape learning improved, but class discrimination did not. Check label mapping/background offset, class support, classifier gradients/loss weighting, ROI samples, and whether one epoch/fold has enough signal.

Drill. Before the event, verify legality and attach/pre-cache the checkpoint with a checksum. During the offline event, load only the local path. If unavailable, use the official architecture with both weights set to `None`; do not trigger a download.

Day 10

10A. Average aligned logits/probabilities for classification; resize and average semantic logits before argmax; use class-aware NMS/box fusion for detection; associate same-class overlapping instances then combine mask probabilities for instance segmentation.

10B. `V`, `I`, `I`, `V`, `I`.

Final-hour discussion. Preserve at least 15–20 minutes for submission generation, reopening, decoding, ID checks, and recovery. Do not begin an unvalidated model/training change in the final minutes.

Competition morning — one-page checklist

Before training

- ☐ Read metric and submission specification again
- ☐ Confirm allowed imports, internet, and checkpoint rules
- ☐ Fix stable label mapping and background convention
- ☐ Audit files, shapes, dtype, IDs, labels, empties, duplicates
- ☐ Persist leakage-safe folds by stable ID/group
- ☐ Visualize raw and transformed targets
- ☐ Run one forward/loss smoke test

During experiments

- ☐ Change one hypothesis at a time
- ☐ Keep split, initialization, epochs, and exposure aligned
- ☐ Record all loss components and runtime
- ☐ Cache complete OOF predictions once
- ☐ Report main, per-class, slice, and fold metrics
- ☐ Inspect high-confidence errors and annotation problems
- ☐ Save config, mapping, fold, and checkpoint together

Before submission

- ☐ Load each intended checkpoint in eval mode
- ☐ Apply deterministic test preprocessing
- ☐ Ensemble with a task-appropriate rule
- ☐ Apply thresholds selected only from OOF
- ☐ Preserve test IDs and required order
- ☐ Verify shapes, labels, ranges, row counts, NaN/Inf
- ☐ Decode several output rows/boxes/masks back to images
- ☐ Reopen the final file and run the submission checker
- ☐ Keep a known-good fallback submission

My personal defaults

Validation split: _____

Fast baseline: _____

Primary metric: _____

Fallback model: _____

Maximum safe image size/batch: _____

Final-hour cutoff rule: _____

Final reminder

The winning move is often not a novel architecture. It is the correct target contract, a split that does not lie, an official model that runs within the rules, an error report that identifies the next useful change, and a submission that faithfully preserves every ID.